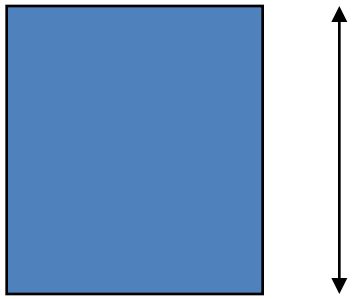
Overview of main concepts

Physical limits: how fast can a serial computer be?

1 Tflop/s, 1 Tbyte sequential machine



r = 0.3 mm

space = velocity x time
r c $1/10^{12}$

- Consider the 1 Tflop/s sequential machine:
  - Data must travel some distance, r, to get from memory to CPU.
  - Go get 1 data element per cycle, this means 10^{12} times per second at the speed of light, $c = 3 \times 10^8$ m/s. Thus $r < c/10^{12} = 0.3$ mm.
- Now put 1 Tbyte of storage in a 0.3 mm 0.3 mm area:

  in fact $0.3^2\ mm^2/10^{12} = 9\ 10^{-2}\ 10^{-6}\ m^2/10^{12} =$

  $9\ 10^{-20}\ m^2 = (3\ 10^{-10})^2\ m^2 = 3^2\ A^2$ →
  - Each byte occupies less than 3 square Angstroms, or the size of a small atom! (1 Angstrom = 10^{-10} m = 0.1 nanometer)

No choice but parallelism !!!

IBM stores one bit of data on a SINGLE atom: Breakthrough may lead to credit card-sized devices capable of holding the entire iTunes library of music

- **Prototype device uses world's smallest ever magnet using one atom of holmium**
- **Stores 1,000 times more information in the same space as existing technology**
- **Could lead to creation of radically smaller hard drives with vastly greater storage**
- **Entire 35 million song iTunes library could be saved to a pocket sized drive**

By TIM COLLINS FOR MAILONLINE
PUBLISHED: 14:35 GMT, 9 March 2017 | **UPDATED:** 14:35 GMT, 9 March 2017

Share

329 shares

31 View comments

Imagine one day being able to carry vast libraries of data, currently stored in rooms full of servers, in your pocket.

That's the dream of a team of IBM scientists who have built what they say is the

Site Web Enter your search

Like Daily Mail

Follow Daily Mail

Download our iPhone app

Today's headlines Most

'Unnatural structures' on the m evidence of an alien base that N hiding, claim conspiracy...

Raising Ramses: Archaeologists year-old statue of Egyptian Pharac considered 'one of the most...

Will Apple release a trio of new WEEK? New 10.5-inch and 12.9 Pro models set for...

Beware of 'time travel' texts from Experts warn strange messages scam

NASA tests radical electric prop system that could take man to M single tank of fuel

Samsung's Galaxy S8 will be a FL

Automatic parallelism in modern machines

- **Bit level parallelism**: for floating point operations, etc. (es. bit shifts for multiplications, etc)
- **Instruction level parallelism (ILP):** execution of multiple instructions per clock cycle
- **Memory system parallelism**: overlap of memory operations with the computation
- **OS parallelism**: multiple tasks run in parallel (eg, threads)

There are obviously limits to these!

Hence, to achieve high performance, the programmer needs to **identify**, **schedule** and **coordinate**

the **<u>parallel tasks and data</u>**

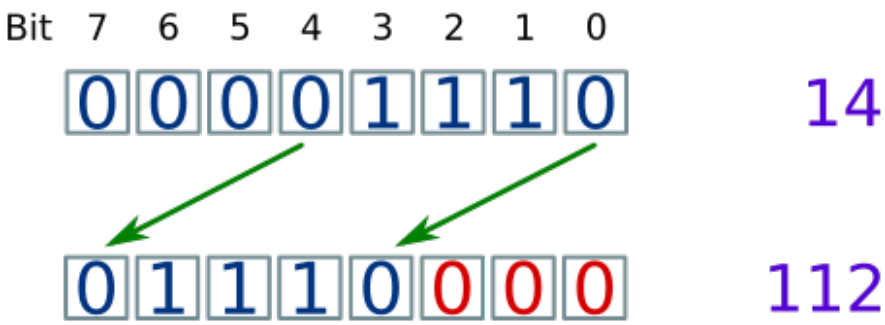
and

Become a Parallel Computing programmer!

Bit level parallelism

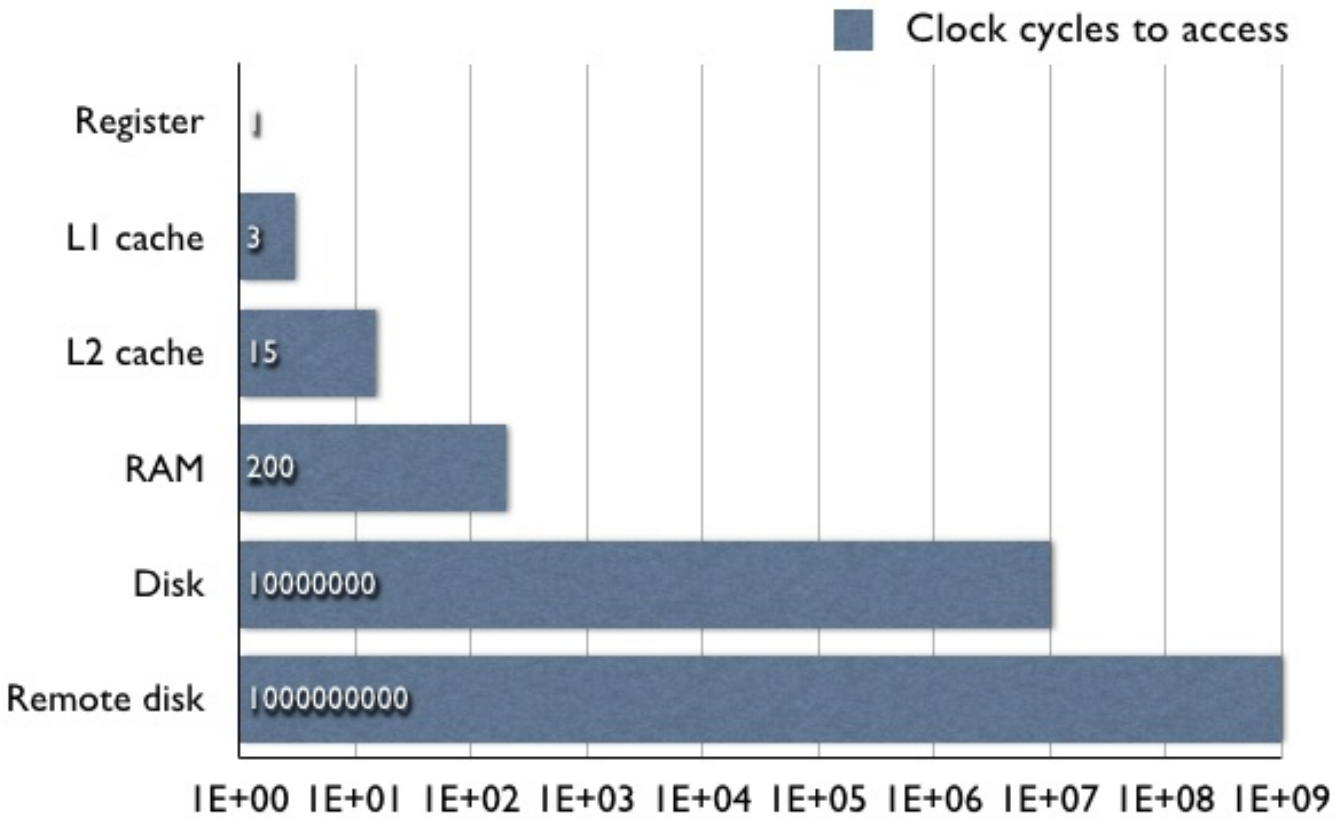
Binary	Decimal
0 0 1 1 0 0 1 0	50
← 2 bits	
1 1 0 0 1 0 0 0	50 x 4 = 200
1 0 0 1 1 0 1 0	-102
→ 1 bit	
1 1 0 0 1 1 0 1	-102 / 2 = -51

Each left shift is multiplication of a power of 2



14 x 8 = 112

Memory Hierarchy



Processors vs. memory access

In 1986
Clock cycle ~ 120 nanoseconds
DRAM access time ~ 140 nanoseconds
» 1:1 ratio ☹

In 1996
Clock cycle ~ 4 nanoseconds
DRAM access time ~ 60 nanoseconds
» 20:1 ratio ☹☹

In 2002
Clock cycle ~ 0.6 nanoseconds
DRAM access time ~ 50 nanoseconds
» 100::1 ratio ☹☹☹

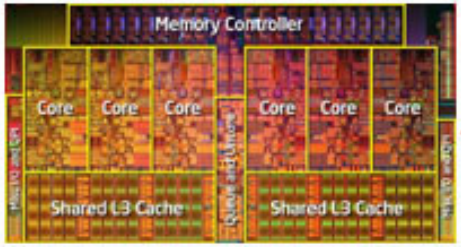
From 2003, in fact, first Dual Core processor .....(by Power PC, not Intel!)

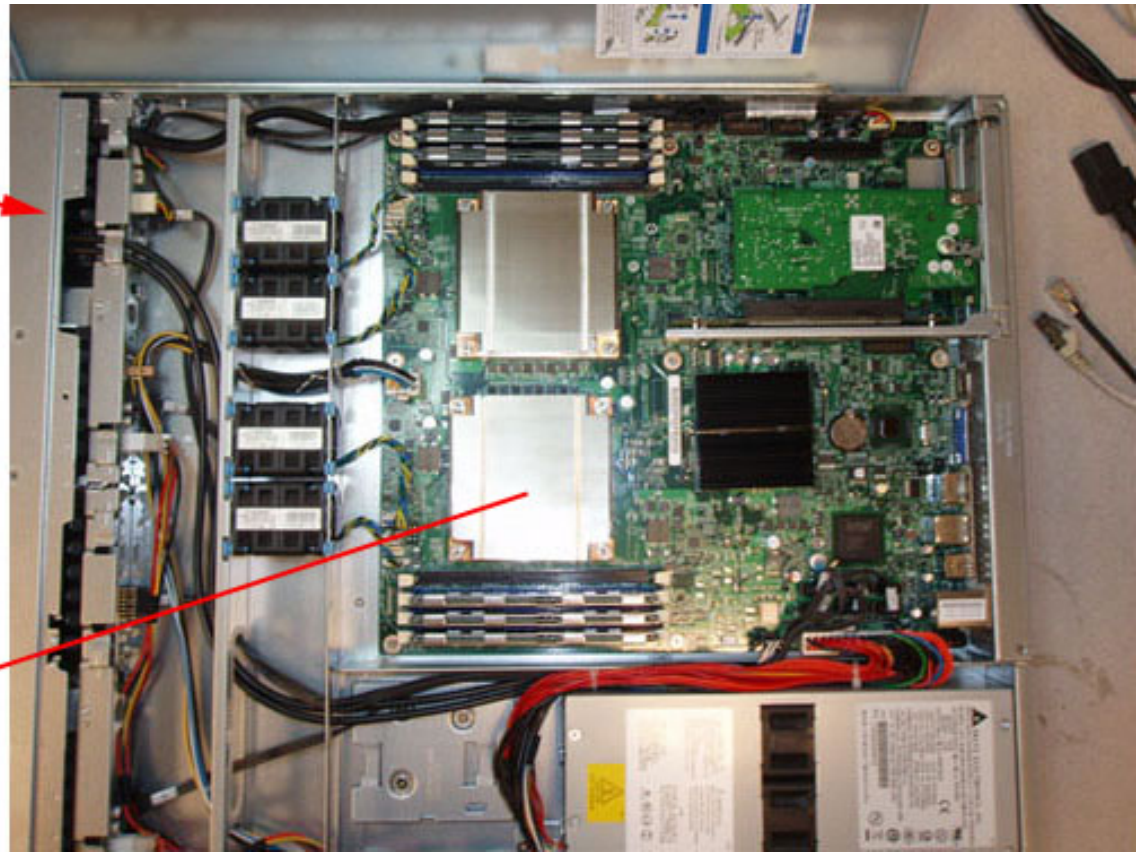
Supercomputers, CPUs, Nodes, Processors, Sockets

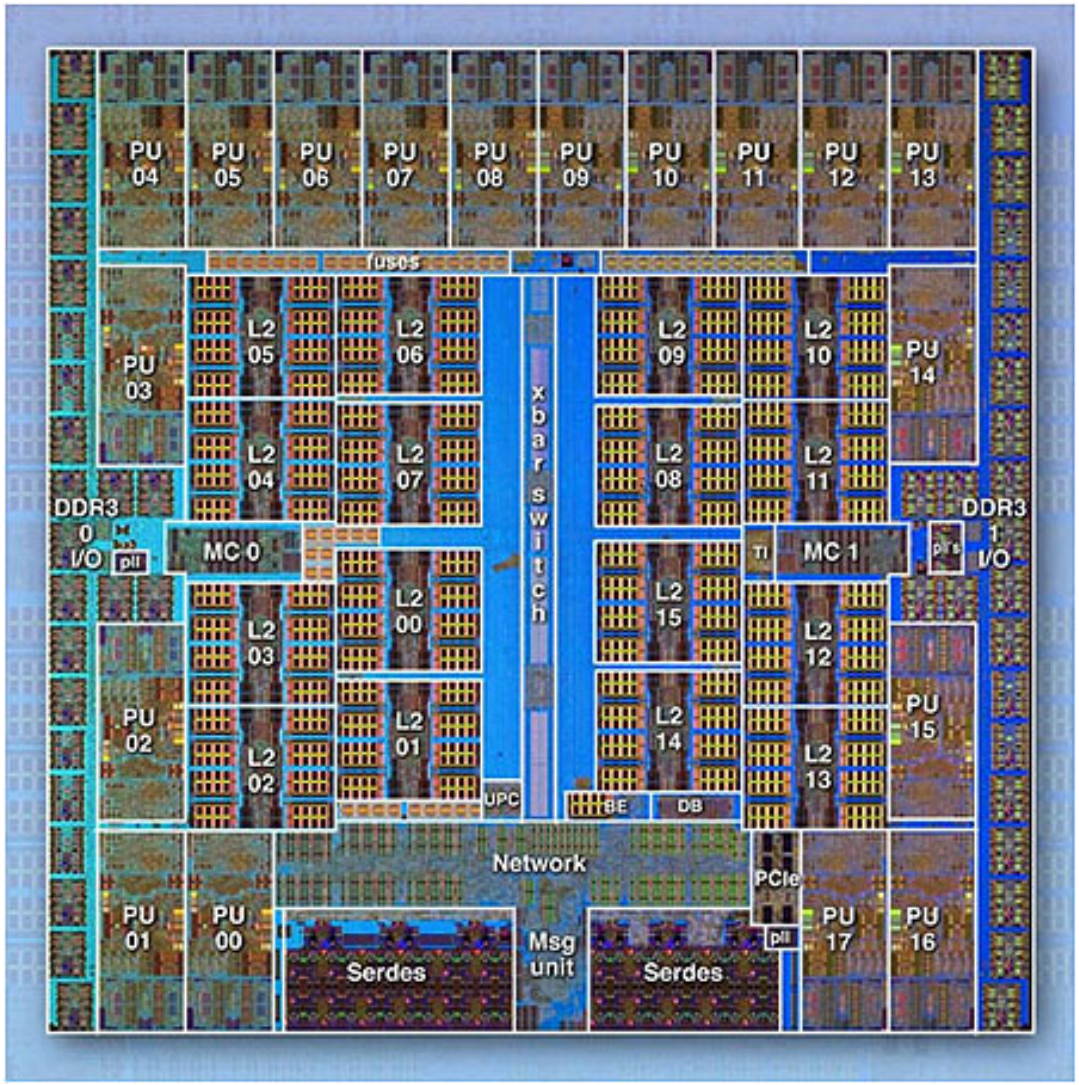
Supercomputer - each blue light is a node

Node - standalone Von Neumann computer

CPU / Processor / Socket - each has multiple cores / processors.







IBM BG/Q Compute Chip with 18 cores (PU) and 16 L2 Cache units (L2)

Why do fast (parallel) machines run slow?

Latency

Waiting for access to memory or other parts of the system

Overhead

Extra work that has to be done to manage program concurrency and parallel resources the real work you want to perform

Starvation

Not enough work to do due to insufficient parallelism or poor load balancing among distributed resources

Contention

Delays due to fighting over what task gets to use a shared resource next. Network bandwidth is a major constraint.

Supercomputers

Check this out

www.top500.org

… after checking this …

http://en.wikipedia.org/wiki/Teraflops_Research_Chip

Leonardo Supercomputer #4! (2025)

What is a Benchmark?

- Comparison of different systems in solving "something hard" as "dense systems of linear equations"

- The comparison involves different computers

- Famous Benchmarks :
  - LINPACK: Linear equation resolver
  - Typical floating-point operations and use of data structures (vector/array)

Performance

- The measurement of the performance of a certain parallel machine makes sense for:
  - The magnitude of the “bigger” problems (ie the size of the problem that "make sense")
  - The performances reflect the largest problem that can be executed on a given machine

- $\mathbf{R_{max}}$ – The performance in Tflops of the largest problem that runs on the machine
- $\mathbf{N_{max}}$ – The size of the largest problem that runs on the machine
- $\mathbf{N_{1/2}}$ – the size where half Rmax is obtained
- $\mathbf{R_{peak}}$ – Theoretical peak performance (in Tflops) of the machine

TOP 7 – November 2024

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
5	**HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461
6	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899

Terminology

Task

A discrete logic section of computational work. A task is typically a program or "piece" of the program that is executed by a processor

Parallel Task

A task that can be executed by multiple processors in a "safe" (i.e., provides correct results)

Serial Execution

Execution of a program sequentially, one instruction at a time. In the simplest sense, this is what happens using a machine with a processor. However, virtually all parallel tasks have sections of a parallel program to be executed in a serial fashion

Parallel Execution

Execution of a program consists of several tasks with each task that is "able" to carry out the same or different instructions at the same time

Terminology

Shared Memory

From a hardware point of view, it describes an architecture where each processor has direct access (**usually via bus**) to a common physical memory.

From a software point of view, it describes a model where parallel tasks have the **same "picture" of memory**, and can access and address the same logical memory locations, regardless of where it is located

Distributed Memory

From a hardware point of view, it refers to a memory access based on **network**, for a not in common physical memory .

From a software point of view, the task can only " see " from a logical point of **view the local memory** of the machine, and can use communications to access the memory of other machines where other tasks are running

Communications

Parallel tasks typically need to exchange data. There are several ways to achieve this, for example through a shared memory bus , or via a network. However , the exchange of data is reported as effective ***communication*** , regardless of the method used

Terminology

Synchronization
The coordination of parallel tasks in real-time, often associated with communications. It is often implemented by establishing a sync point in an application where a task can not continue until another task (or more) achieve the same (or logically equivalent) point. Synchronization usually involves the expectation of at least one task, and can therefore cause an increase in the "wall - clock" of parallel applications

Parallel Overhead
Amount of time taken to coordinate parallel tasks , in contrast to useful work. The determining factors are:

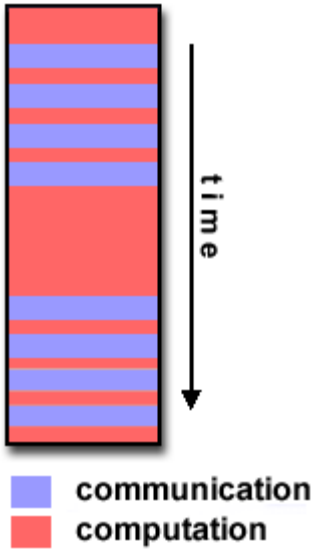
1. Task start-up time
2. Synchronizations
3. Data Communications
4. Software overhead imposed by parallel compilers , libraries , tools, operating systems, etc.
5. Task termination time

Granularity
In parallel computing, granularity is a qualitative measure of the relationship between computation and communication.
Coarse : relatively large portions of computational work are done between communication
Fine : relatively small portions of computational work are done between communication

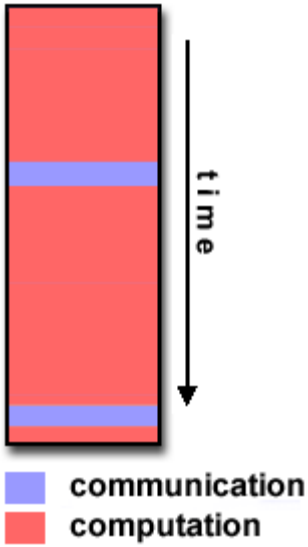
Granularity

- Computing / Communication Ratio

Fine-grain Parallelism :



Coarse-grain Parallelism :



Terminology

Massive Parallelism

Refers to hardware that includes a given parallel system consisting of many physical processors. "Many" refers to date in systems with more than 10,000 processors (i.e., GPGPU)

Scalability

Refers to the ability of parallel systems (hardware and / or software) to allow a **proportional increase** in the speed-up with the increase of processors. Factors that contribute to scalability include:

- Hardware - especially the memory bandwidth-cpu bandwidths and network communications
- The used algorithm!
- The parallel overhead
- Characteristics of your specific application and coding

Scalability

Remember

"In theory, there is no difference between theory and practice. In practice there is."

Yogi Berra

gcc/g++ -O compiler optimizations

option	optimization level	execution time	code size	memory usage	compile time
-O0	optimization for compilation time (default)	+	+	-	-
-O1 or -O	optimization for code size and execution time	-	-	+	+
-O2	optimization more for code size and execution time	--		+	++
-O3	optimization more for code size and execution time	---		+	+++
-Os	optimization for code size		--		++
-Ofast	O3 with fast none accurate math calculations	---		+	+++

+increase ++ increase more +++increase even more
-reduce --reduce more ---reduce even more

gcc/g++ -O compiler optimizations

- By changing compiler optimizations, different execution times can be obtained for the **serial** version
- … but also for the **parallel** one!

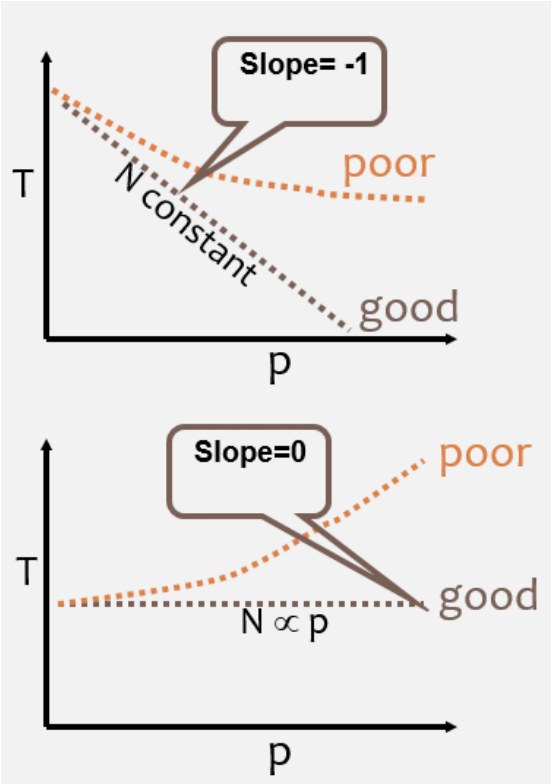
- **Take different timings with different compiler options to evaluate speedups, etc…**

Scalability:
The main reasons for Parallel Computing

- Increase the size and complexity of problems that can be solved
  - Big problems may not be contained in a sequential computer's memory
    - Distribute the problem on many computers memories (**strong scaling**)
  - Bigger problems may not be solvable on sequential computers in a reasonable time
    - Decompose in smaller problems (**weak scaling**)
- Reduce the time to solve a problem
- Solve bigger problems, more quickly

Scalabilty

- **Strong scaling:**
  - The total problem size N stays fixed as more processors P are added.
  - Goal is to run the same problem size faster
  - Perfect scaling means problem is solved in 1/P time (compared to serial)
- **Weak scaling:**
  - The problem size *per processor* stays fixed as more processors are added.
  - Goal is to run larger problem in same amount of time
  - Perfect scaling means problem PxN runs in same time as single processor run



Hardware factors play a significant role in scalability:

-Memory-cpu bus bandwidth on an SMP machine

-Communications network bandwidth

-Amount of memory available on any given machine or set of machines

-Processor clock speed

Programming languages adopted in High Performance Computing

- **Message Passing Interface (MPI)**
  - **Distributed memory context:** Explicit sharing of variables from one processor to another, so that each process can manage a memory segment of the program
- **Pthreads/OpenMP**
  - **Shared memory context:** Manages the distribution of process and data in SMP machines, communications occur by means of **shared memory**
- **CUDA/ OpenCL / Sycl**
  - De-facto programming languages for GPGPU based systems

Classification of Parallel Computers

- Purely functional, by Flynn:
  - SIMD, SIMD, MISD, MIMD
  - Structural
    - Memory based
      - Shared memory, distributed memory
    - Based on the interconnection network
      - Bus, crossbar, direct and indirect networks

Flynn's Taxonomy

In 1966, Michael J. Flynn classifies Computing Systems according to the multiplicity of the instruction stream and the data stream that can be handled

SISD (Single Instruction Single Data)

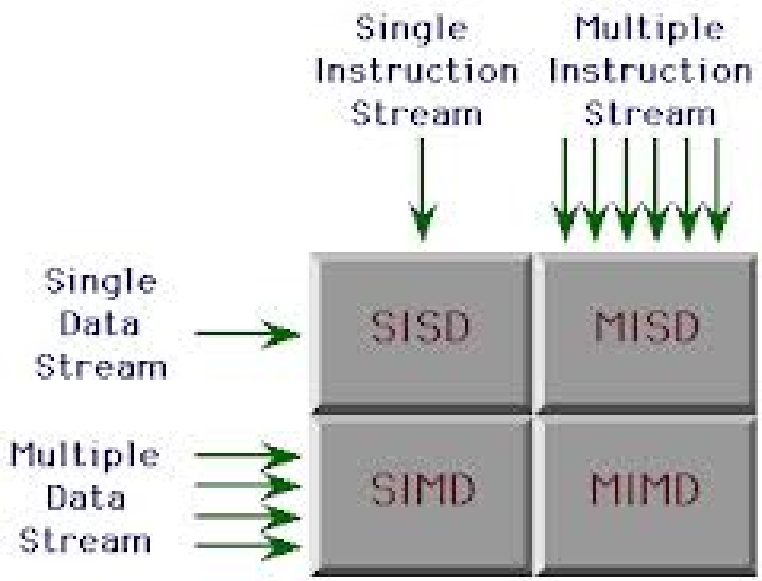
SIMD (Single Instruction Multiple Data)

MISD (Multiple Instruction Single Data)

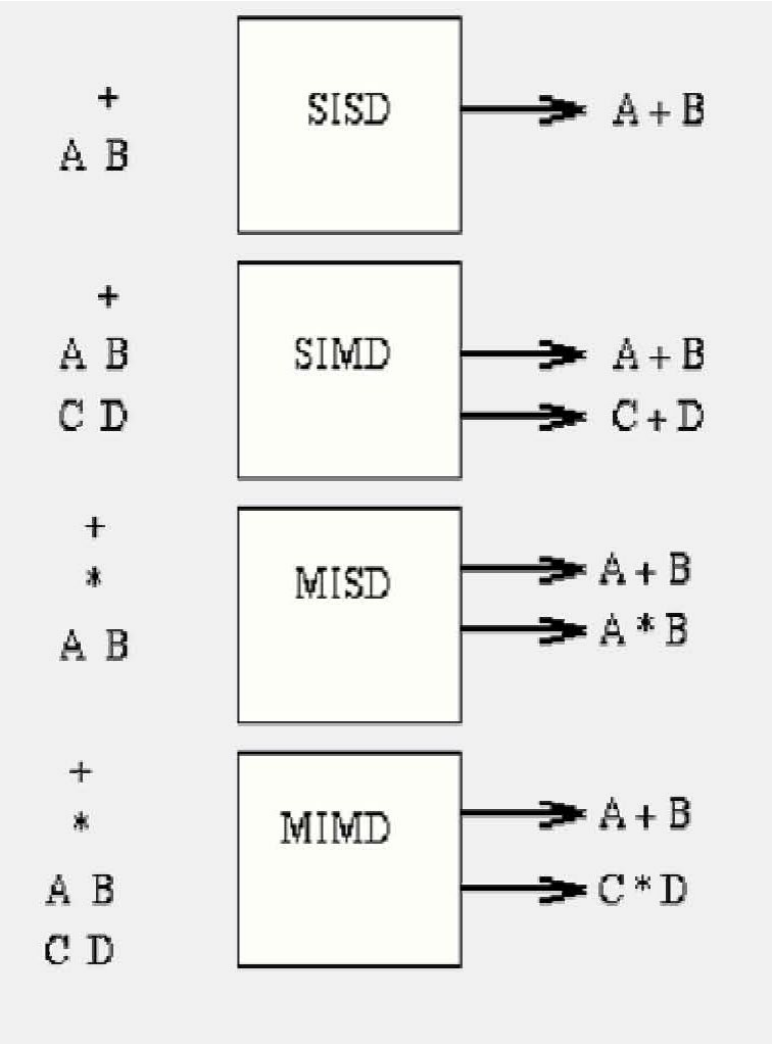
MIMD (Multiple Instruction Multiple Data)

Flynn's Classification

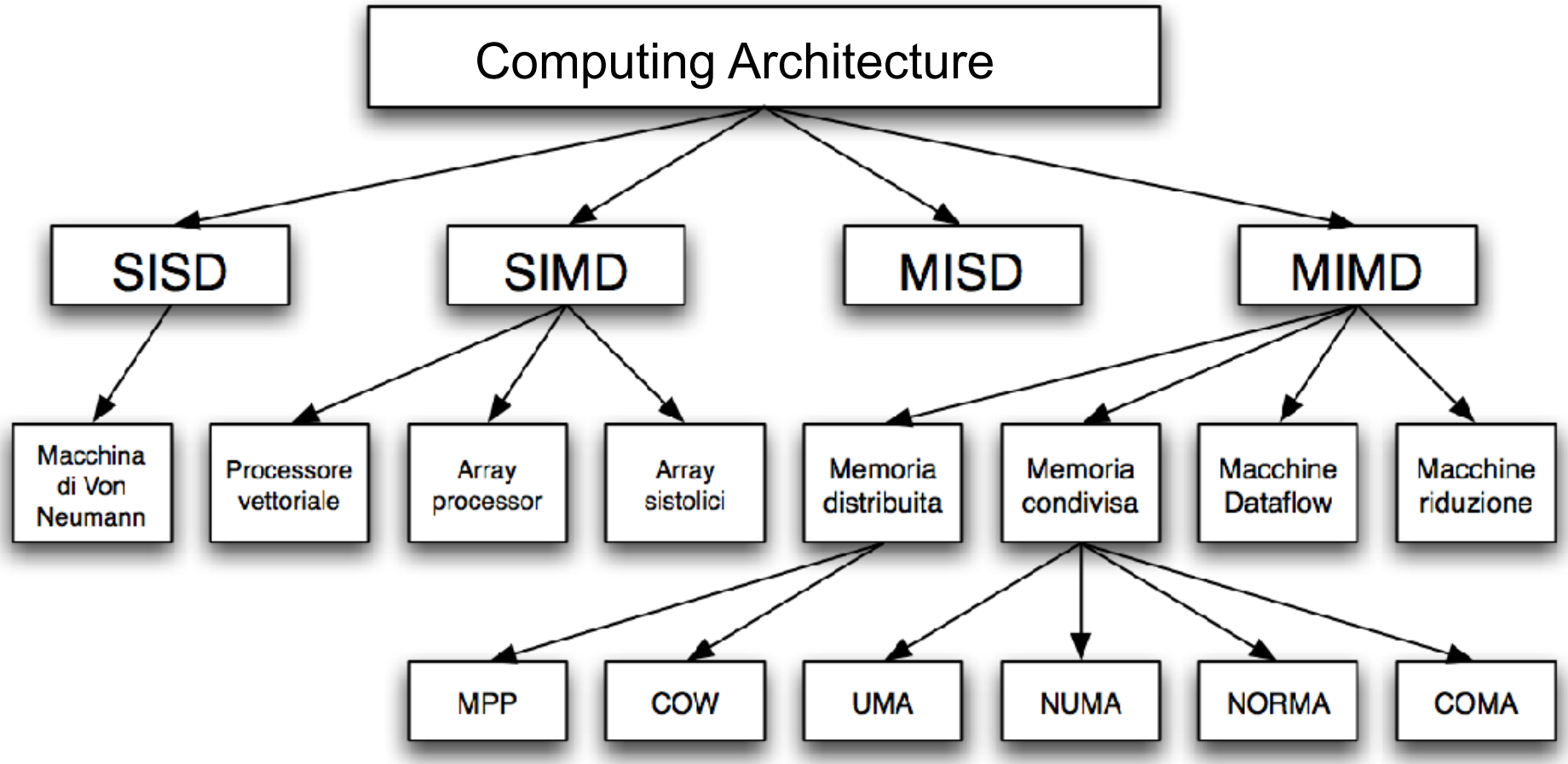
- **Michael Flynn** introduced in 1966 the following **taxonomy**, where "interesting" parallel computer classes are **MIMD** (Multiple Instruction stream-Multiple Data Stream) and **SIMD** (Single Instruction stream-Multiple Data Stream)



Potentialities of the 4 models



Flynn's Taxonomy

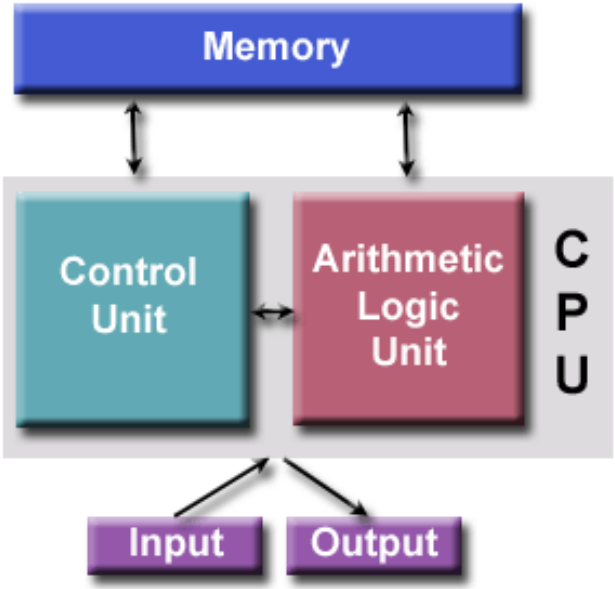


von Neumann Architecture (SISD)

- For over 50 years, every computer has adopted a common model of architecture known as the von Neumann computer, named after the Hungarian mathematician John von Neumann
- A von Neumann computer uses the concept of "stored-program". The CPU executes a stored program that specifies a sequence of reads and writes to memory

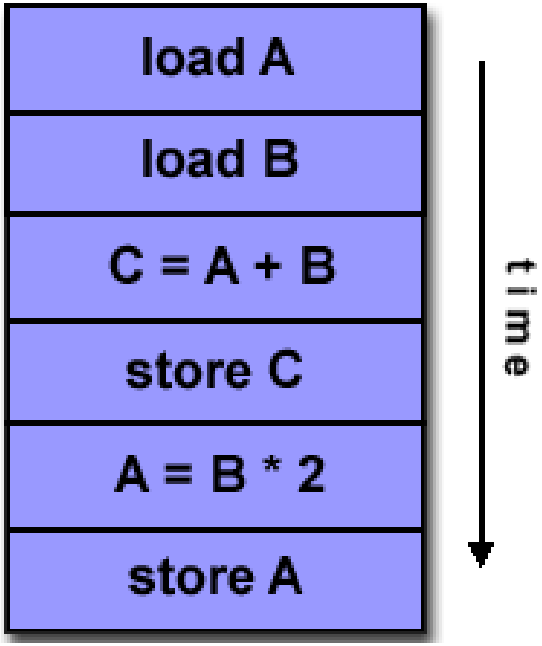
Basic architecture:

- The **memory** is used to store both program instructions and data
- The **instructions** in the program encode data which tell the computer what to do
- **Data** is simply information that is used by the program
- A **Central Processing Unit (CPU)** fetches the information and / or data from memory, decodes the instructions and executes them sequentially



Single Instruction, Single Data (SISD)

- A sequential (non-parallel) calculator
- Single instruction: only one instruction stream is executed during a machine cycle
- Single data: only one data stream is used as input during a machine cycle
- Deterministic execution
- Examples: the majority of PCs, workstations, and mainframes single CPU



SISD Machines

UNIVAC

CRAY 1

PDP1

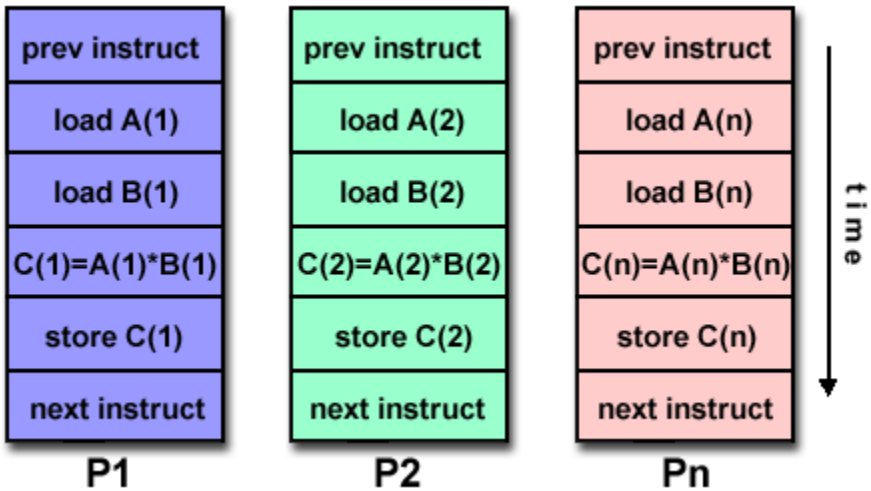
NOTEBOOK

Single Instruction, Multiple Data (SIMD)

•**Single instruction**: All computing units execute the same instruction at each cycle clock

•**Multiple data**: Each processing unit element can operate on a different date

•Typically, this machine has as **instruction dispatcher**, an internal high bandwidth network and a large number of **small units** for instructions



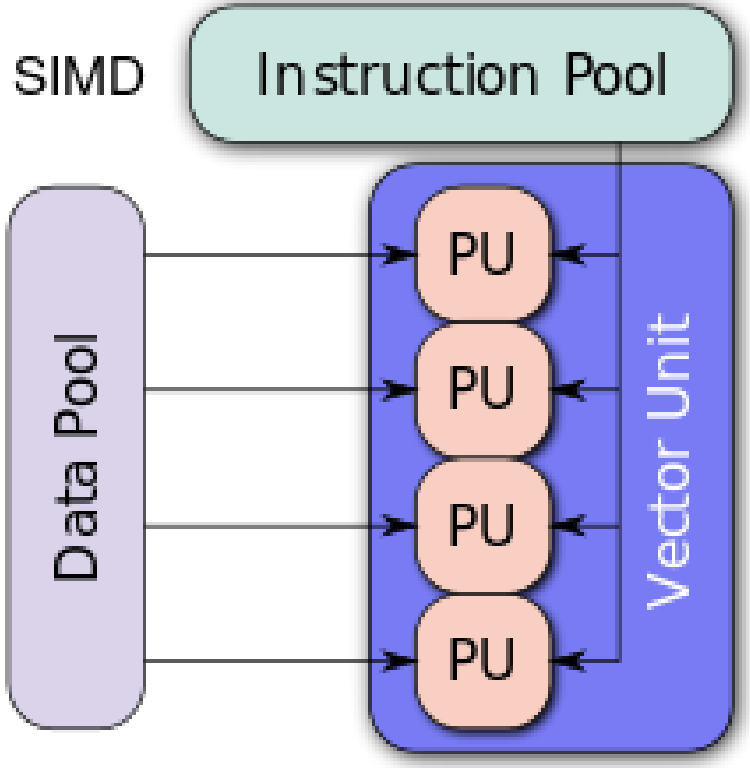
•Used mainly for "specialized" problems characterized by a high degree of regularity, as image processing (eg. GPU cards)

•Synchronous (**lockstep**) and deterministic execution

•Examples:
- Processor Arrays: Connection Machine CM-2, MasPar MP-1, MP-2
- Vector Pipelines: IBM 9000, Cray C90, Fujitsu VP, NEC SX-2, Hitachi S820
- GPU processors of standard computers (NVIDIA, ATI, etc.) or dedicated (TESLA)

SIMD

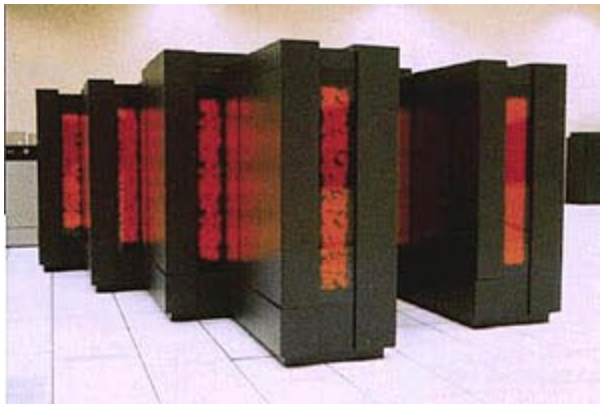
- **Synchronous operations**
  - At each clock step all processors execute the same instruction on different data
- **Array processors** (Connection machine, MasPAR MP, etc)
- Pipeline vector computer (Cray 1 & 2)
- **SIMD machines** are suitable for solving **data-parallel problems**, where the same instruction is applied to distinct partitions:
  - Ex: Matrix sum: A + B = C
  - If A [2][2] and B[2][2] are distributed to 4 processors, in parallel we have:
    - A11 + B11 = C11 A12 + B12 = C12
    - A21 + B21 = C21 A22 + B22 = C22



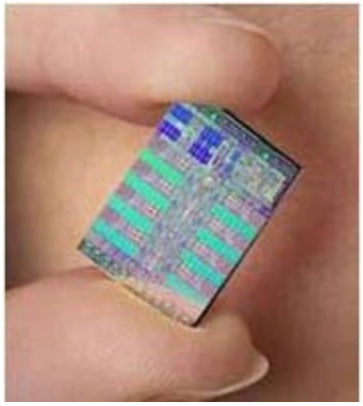
- **GPGPU!!!**

SIMD Machines

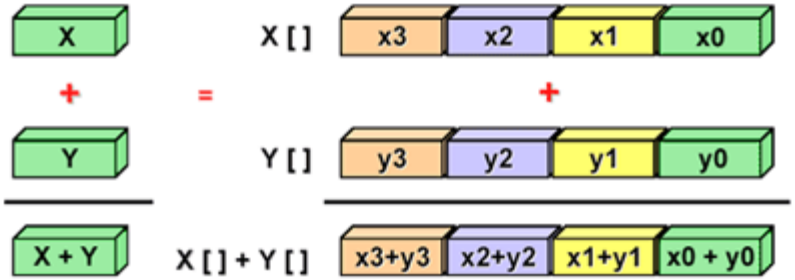
Cray XMP



Thinking Machines (CM2)



Cell Processors (GPU)

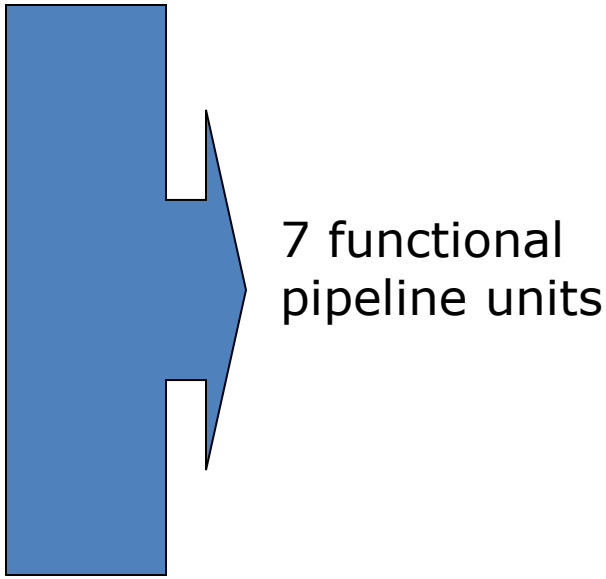


Pipeline Architectures

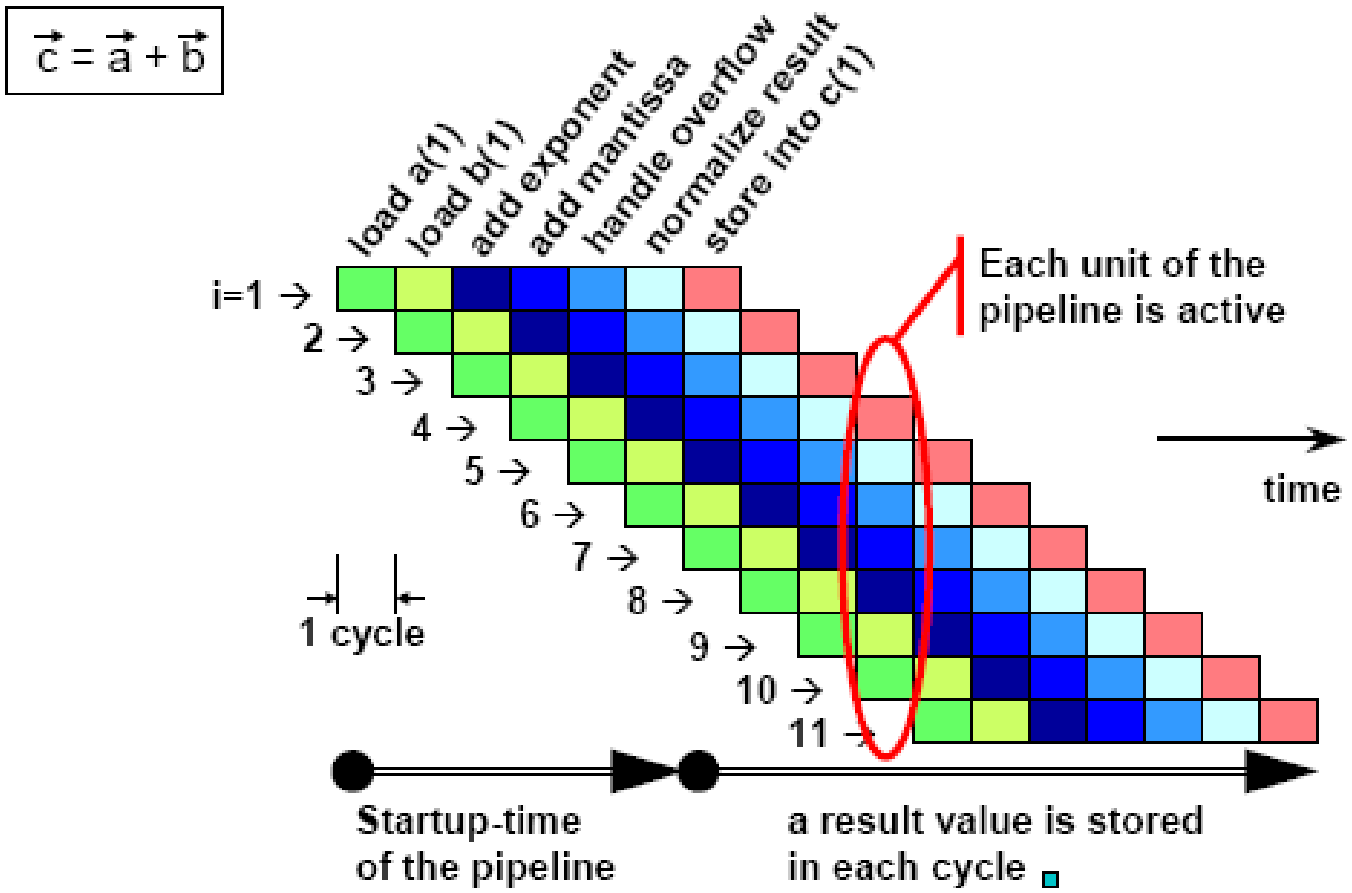
```
float a[100], b[100], c[100];
for (i=0; i<100; i++)
    c[i]= a[i] + b[i];
```

A single instruction can consist in:

1. Fetching operand `a` from memory
2. Fetching operand `b` from memory
3. Exponent summation
4. Mantissa summation
5. Overflow management
6. Normalization
7. Store `c`



Pipeline Architectures



SIMD Programing

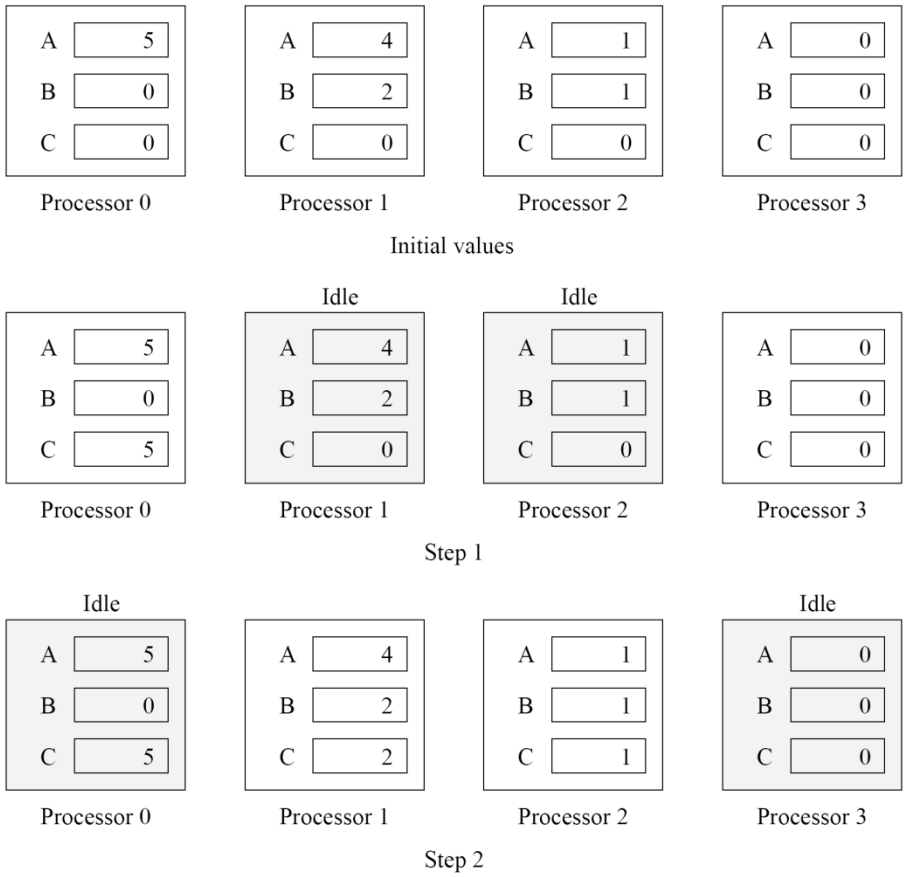
```
for (i=0; i<1000; i++)
      if (y[i]!=0.0)
            z[i]=x[i]/y[i];
      else
            z[i]=x[i];
```

Assuming an element for each processor, running synchronous (in **lockstep** - at each step each processor does the same thing):

```
1.   if (local_y!=0.0)
2.   if (local_y!=0.0) then local_z = local_x / local_y
     if (local_y==0.0) then do nothing
3.   if (local_y!=0.0) then do nothing
     if (local_y==0.0) then local_z = local_x
```

```
if (B == 0)
        C = A;
else
        C = A/B;
```

(a)



(b)

Figure 2.4 Executing a conditional statement on an SIMD computer with four processors: (a) the conditional statement; (b) the execution of the statement in two steps.

Multiple Instruction Single Data (MISD) ???

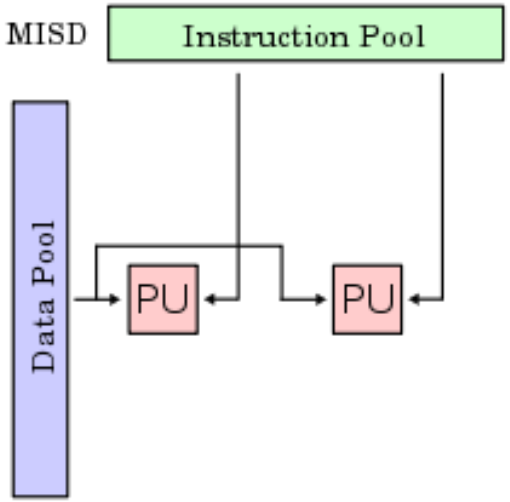
- Few real and practical examples exist

Some examples might be:

- Multiple frequency filters that operate on a single flow signal

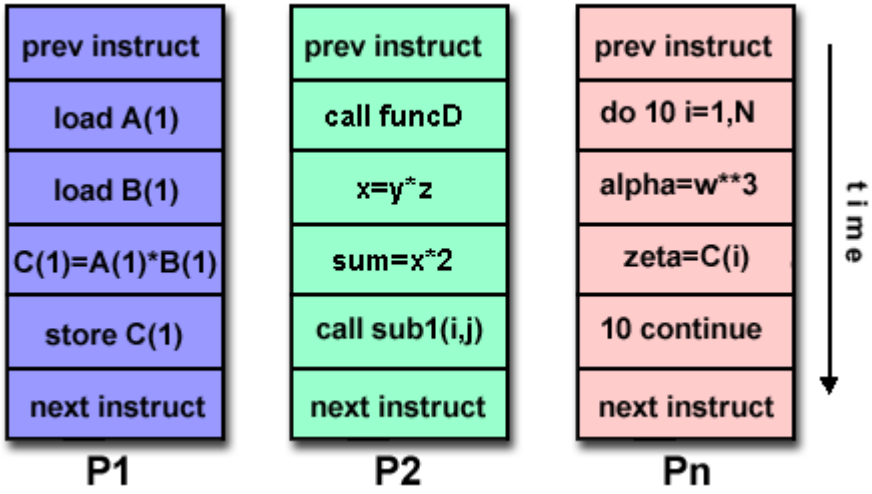
- Multiple encryption algorithms that try to crack a single message

...



Multiple Instruction Multiple Data (MIMD)

- The most common type of parallel computer !



- **Multiple Instruction**:
Each processor can execute a different stream of instructions

- **Multiple Data**:
Each processor can work with a different data stream

- Execution can be synchronous or asynchronous, deterministic or non-deterministic

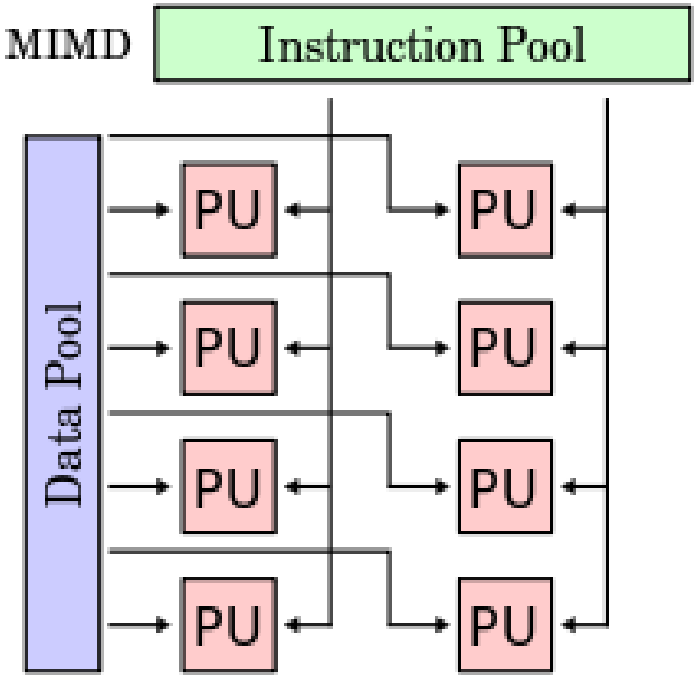
- **Examples**: current supercomputers, "grid" of parallel computers based on network computers and multi-processor SMP, all existing PC / notebook (eg Dual Core i7)

MIMD
Multiprocessor, Multicomputer

- Each processor operates in **asynchronous** mode under the program control and on different data

- **Shared memory MIMD computers** are known as **multiprocessor** ore tightly coupled machines (ex: all current multi-core computers, etc)

- **Network based MIMD computers** are known as **multicomputer** or loosely coupled machines (es: Transputers, CRAY T3E, Beowulf clusters, etc)



MIMD Machines

Compaq/HP Alpha server

AMD Opteron

IBM Blue Gene

MIMD Paradigms

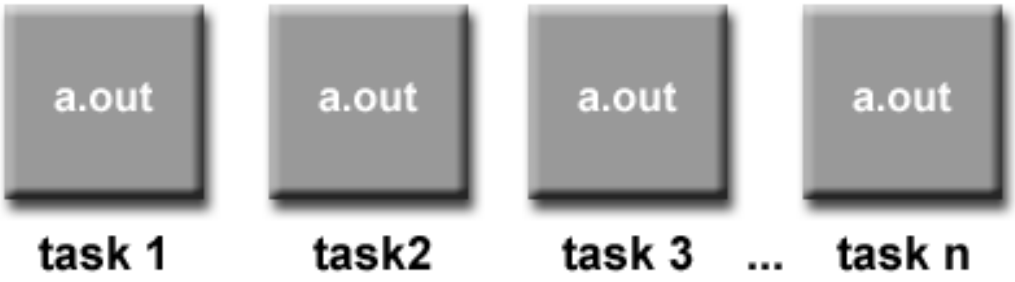
- **MPMD** - Each processor has a different program to be executed

- **SPMD** - Each processor has the same program running, although independent and asynchronous. MPI (Message Passing Interface) is based on this paradigm

- However, with branching techniques, the model can be efficiently emulated on MPMD models SPMD

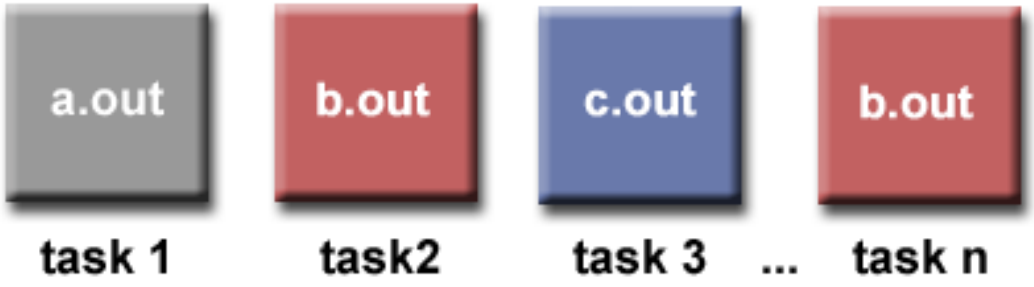
SPMD Model

- The **Single Program Multiple Data (SPMD)** model is a high-level parallel programming model that is based on the models mentioned previously
- A **program** (task) is performed by all individual processes simultaneously
- At any given time, processes can execute the same instructions or other instructions.
- **SPMD** programs usually have a specific logic that allows them to run, by "**branching**", only those parts of the program for which they are designed. That is, the process does not need to run the entire program, but only part of it. **MPI adopts this model!**
- All tasks usually work on different data



MPMD Model

- As the SPMD model, the **Multiple Program Multiple Data (MPMD)** model is a parallel programming model that can be constructed on any combination of the previous models
- MPMD typical applications consist of multiple executable programs. While the parallel application is running, each task can run the **same program or a different one**
- All tasks usually work on different data

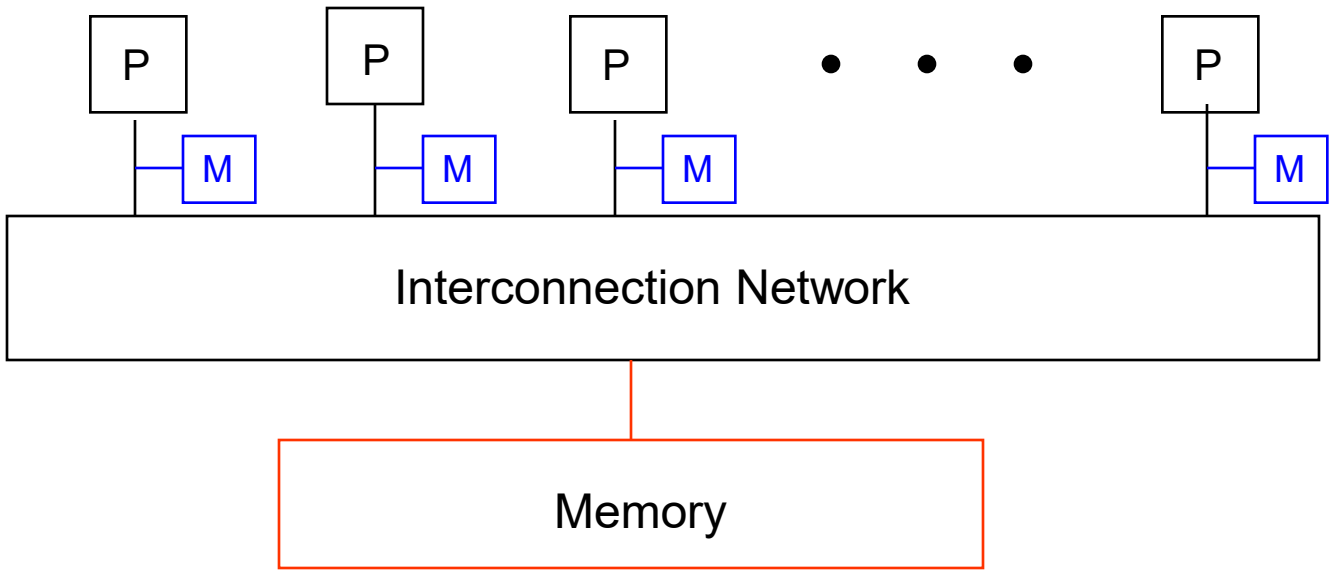


Structural classification based on memory

- In a parallel system, **2 typical memory configurations** are possible:

- **Shared-memory**
  1. A single memory is present, where all **processing elements (PE)** can access
  2. Message exchanges between PEs occurs by writes in this memory
  3. If the number of PEs is high, memory can become a **bottleneck**
- **Distributed memory**
  1. Each PE has its local memory
  2. Communication occur by means of message exchanges (**Message Passing**)

Generic Parallel Architecture



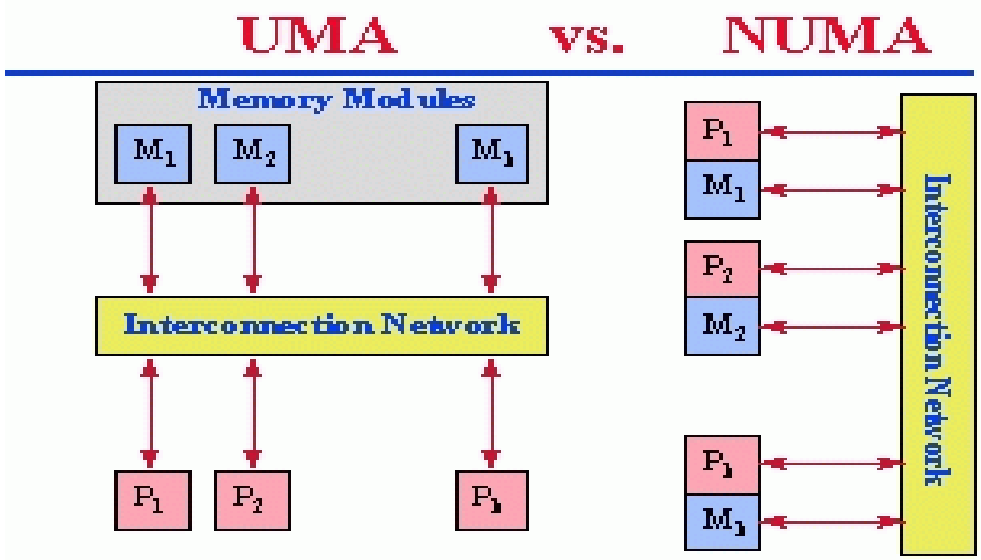
Where is memory physically located?

Shared memory vs Message passing

- In both SIMD and MIMD machines, **cooperation** is achieved by means of **data exchange**
- This can occur by:
  - Using **Shared memory (SM)** and shared variables
  - Using **Distributed memory (DM)** and message passing
- In the case of **SM,** we have a **single address space**, where all processors can access data
  - Problems linked to scalability, partially solved by cache usage
  - Cooperation suitable only for few processors (2-32 MIMD multiprocessors)
- In the case of **DM**, we have a **private addressing to each processor**, so message passing is necessary for communication
  - Typical modality for many MIMD computers and workstation based clusters
  - If necessary, a single address space can be **emulated** by a suitable software (e.g. Virtual Shared Memory)

Shared Memory Multiprocessors

- Load/Store operations access directly all memory locations of the system memory
  - Simple mechanism for communication and sharing
    - Transparent respect to location
    - Same operations available from uni-processors
- Central memory or distributed «near» processors
- Cost of Uniform Memory Access (UMA) or Non Uniform Memory Access (NUMA)



Structural classification on interconnection basis

- Typical interconnection network include:
  - Single Shared bus
  - Multiple bus
  - Crossbar
  - Ring
  - Hypercube
  - tree